

ITSS SOFTWARE DEVELOPMENT/ SOFTWARE DESIGN AND CONSTRUCTION

Lab 04: Sequence Diagram

Lecturer: NGUYEN Thi Thu Trang, trangntt@soict.hust.edu.vn

1. SUBMISSION GUIDELINE

To submit this lab, you and your teammate have to push all the work to the group GitHub repository, following the naming convention **MSTeamsName-TeamNumber** (e.g., TKXDPM.VP.20251-01).

For this lab, you are required to check out a branch named **release/lab04**, contains all tasks related to sequence diagrams.

The deadline for submission will be updated by the lecturer or the teaching assistant. Therefore, you need to pay attention to avoid late submissions.

2. OVERVIEW

With the completion of the “Software Requirement Analysis” document, you have finished the requirement analysis process of the software development process. Starting from this lab, you will move on to the **architectural design** process. This process focuses on building the overall architecture of the software based on the requirements analyzed earlier. Its main purpose is to provide a design for the software that implements the requirements and can be verified against them.

To work with the architectural design process, we can use interaction diagrams. In UML, there are two common types of interaction diagrams: sequence diagrams and communication diagrams. Each type has its own characteristics, and the choice of which one to use depends on your needs and objectives.

In this lab, you will start by creating sequence diagrams following the detailed instructions in **Section 3**. The input for these sequence diagrams is the use case specifications that you created in the previous labs. After completing the sequence diagrams, you will gain an overall view of the initial analysis classes, including boundary classes, control classes, and entity classes.

3. SEQUENCE DIAGRAM

First, you should access the following links and read everything carefully to know how to make a sequence diagram with Astah UML:

<https://astah.net/support/astah-pro/user-guide/sequence-diagram/>

<https://www.youtube.com/embed/Qi2CsTY4LSk>

You can also follow some useful tips:

<https://astahblog.com/2015/10/15/search-models-in-diagram/>

<https://astahblog.com/2015/10/28/hide-sequence-message-number/>

<https://astah.net/support/astah-pro/user-guide/diagram-editor/>

3.1. SEQUENCE DIAGRAM FOR USE CASE “PLACE ORDER”

This subsection demonstrates how to create an interaction diagram for use case “Place Order” step by step. In the end of this subsection, we achieve a sequence diagram for use case “Place Order”, which are used to create the communication diagram and analysis class diagram in the next lab.

3.1.1. Find Analysis Classes

From the use case specification, you have already captured the flow of events of that use case. Review this flow carefully to determine the necessary analysis classes. These may include:

- boundary classes (interaction points with the external environment, such as a user interface screen),
- entity classes (used for storing data for long-term purposes),
- control classes (responsible for coordinating and managing the interaction between boundary and entity classes).

Consider use case “Place Order”:

Step 1. Find boundary classes:

- a) User interface classes: CartScreen, DeliveryInfoScreen, InvoiceScreen and SuccessOrderScreen.
- b) System/Device interface classes: None.

Step 2. Find entity classes: Product, Cart, DeliveryInfo, Order, Invoice.

Step 3. Find control classes: PlaceOrderController.

3.1.2. Create new sequence diagram

Now, open the Astah UML software and create a new sequence diagram named “Place Order”. Place all the analysis classes you have identified and related actor(s) into this diagram. Don’t forget to use the correct symbol for each type of analysis class according to UML conventions.

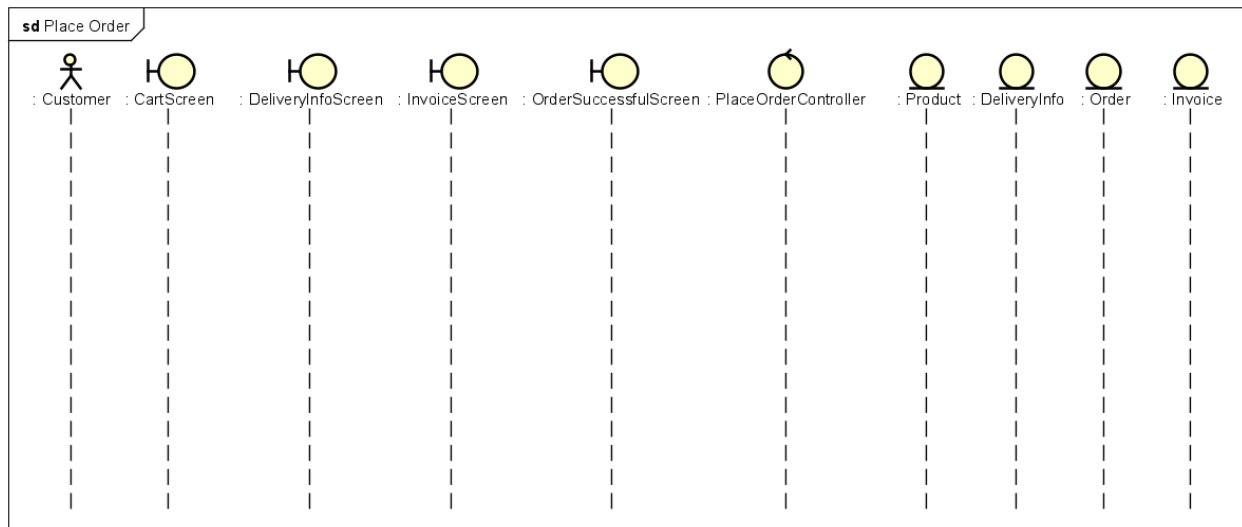


Figure 1. Initial sequence diagram for use case "Place Order"

Question: Why is there a colon ":" in front of each class name?

In UML, when a colon is placed before a class name, it indicates that this is an object (a specific instance) created from that class. These objects may also have their own names, for example, `cartScreen:CartScreen` means `cartScreen` is the specific instance of class `CartScreen`. In a sequence diagram, you are actually working with these specific instances, which can “send messages” to one another.

3.1.3. Distribute use-case behaviour to classes

Note: All below images/diagrams in this guideline are merely examples, *they may be lack or inaccurate with the problem statement*. You need to check carefully to ensure that your sequence diagram(s) are consistent with your use case model.

The very important task you have to do in this step is to decide which objects will “send messages” to which objects.

In use case “Place Order”, to start placing a new order, the customer have to send a “Place Order Request” message to the boundary class that he or she is interacting with. For instance, by clicking the “Place Order” button on the cart screen, all the logic related to order placement will be executed.

Therefore, you can define a message named `requestToPlaceOrder()`, sent from an instance of `Customer` to an instance of `CartScreen`.

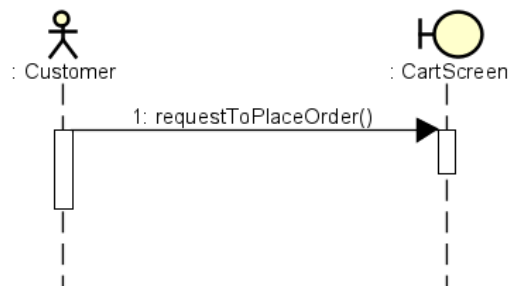


Figure 2. Customer requests to place order

Next, the instance of `CartScreen` will send the message `placeOrder()` to an instance of `PlaceOrderController`, with the argument is the current `cart`.

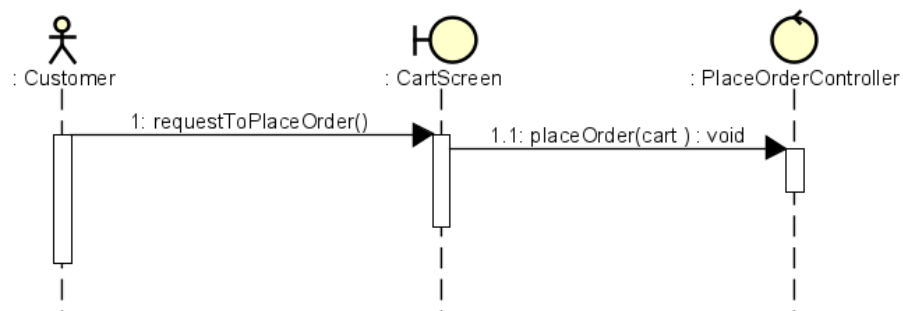


Figure 3. Send message between instances of boundary and control classes

When drawing the sequence diagram, the name of the message needs specifying clearly, and should follow the Java's naming convention for operation (in camel case). In addition, you may define the input parameters/arguments, return type, and return variable for that message, if necessary.

Base Stereotype Constraint Hyperlink	
Name	requestToPlaceOrder
Argument	
Guard	
Return Variable	
Return Type	
Operation	<<Unspecified>> ▼
Property New	

Figure 4. Define message's signature

By following along the flow of events of the use case, you will continue this process and eventually arrive at a complete sequence diagram.

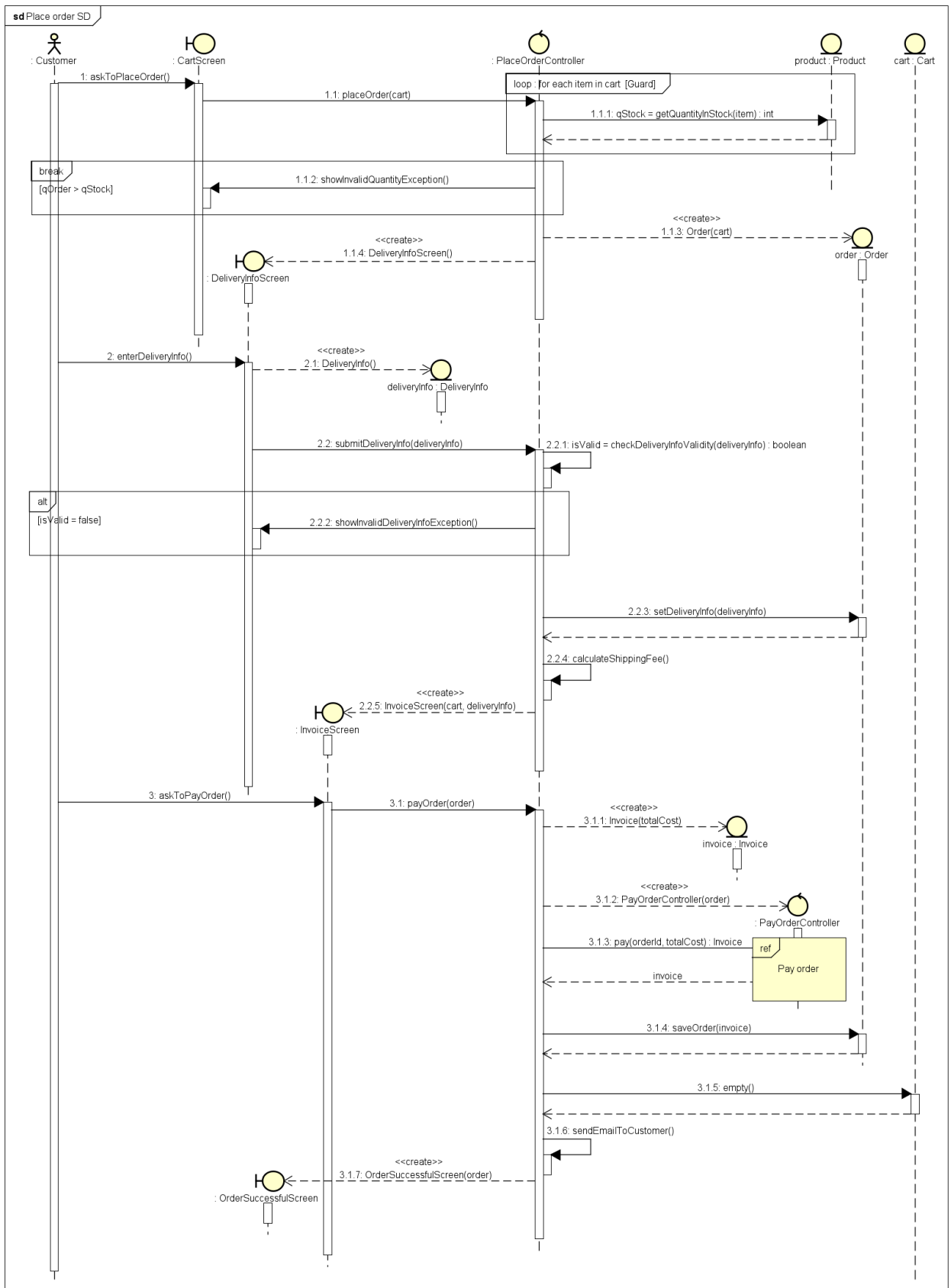


Figure 5. Sequence diagram for use case "Place Order"

Question: How can the alternative flow of a use case be represented in a sequence diagram?

There are several ways to represent the alternative flow of a use case in a sequence diagram. The simplest way is to split it into more than one sequence diagram, where one diagram illustrates the main flow, and the others illustrate the alternative flows.

In addition, Astah UML provides several tools for representing alternative flows (even more than that). What we want to highlight here is the **Combined Fragment**. When you choose to draw a combined fragment in a sequence diagram, you will see that there are many types of operators (as shown in the figure below).

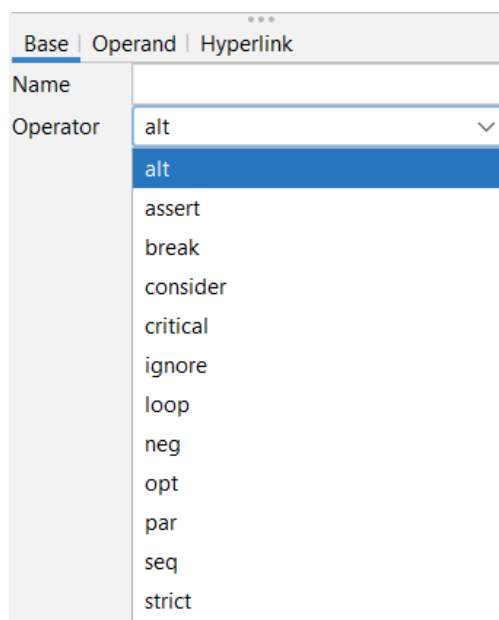


Figure 6. Combined Fragment

In use case “Place Order”, suppose that when checking the stock quantity of the products in the cart, if one product does not have sufficient quantity, you will terminate the use case at that point (or call another use case, depending on the context you define). In this case, we can use the “break” operator to represent it.

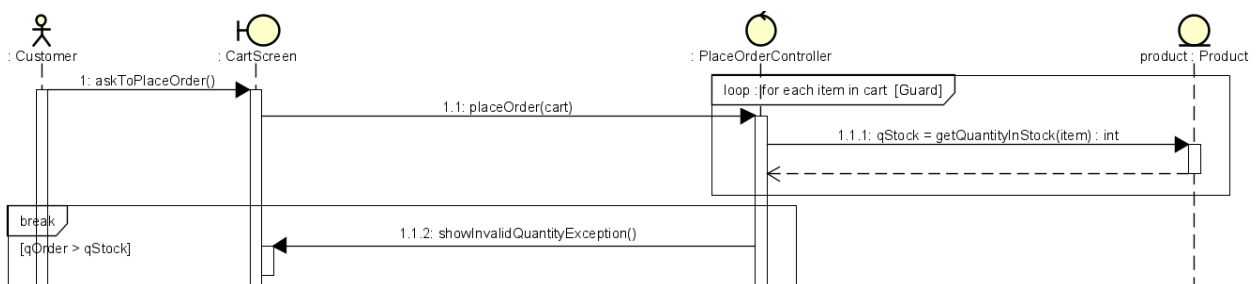


Figure 7. Operator "break" in combined fragment

Question: Should I choose a synchronous or an asynchronous message?

When using Astah UML to draw a message in a sequence diagram, you will see two types of messages: “S” (representing synchronous) and “A” (representing asynchronous). So, when should you choose each type?

A synchronous message is used when the sender waits for the receiver to finish processing and return control before continuing. In other words, the sender is blocked until the operation completes. An asynchronous message, on the other hand, is used when the sender does not wait for the receiver to finish processing. The sender continues its execution immediately after sending the message.

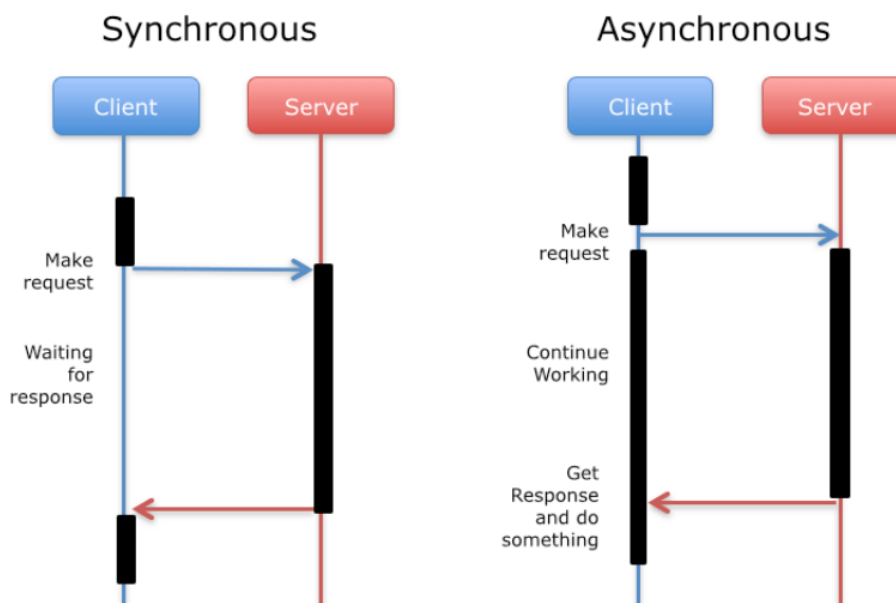


Figure 8. Synchronous vs. Asynchronous

<https://www.linkedin.com/pulse/synchronous-vs-asynchronous-operations-omar-ehab/>

In practice, synchronous messages should be used for operations that must complete before the next step can proceed. For example, validating login credentials must finish successfully before granting access to the system. On the other hand, asynchronous messages are more suitable for operations that can run in parallel or in the background without blocking the main flow. A typical case is sending an email notification after an order has been placed, where the system does not need to wait for the email to be sent before continuing.

Now, it's up to you to decide which type of message to use. At this stage, the choice may still be unclear. However, once you move on to implementation, you will realize

the importance of deciding whether a message should be synchronous or asynchronous.

3.2. SUBMISSION REQUIREMENTS

Now, you will draw sequence diagrams for the use cases assigned to you. You are required to create directories named `ArchitecturalDesign/SequenceDiagram/` to submit your work. You must submit both the Astah project file (.asta) and the exported image file (in PNG or JPG format). Don't forget to name the files according to the corresponding use case.

```
ArchitecturalDesign/  
└─ SequenceDiagram/  
    └─ astah/  
        └─ PlaceOrder.asta  
        └─ ... (put your files here)  
    └─ images/  
        └─ PlaceOrder.png  
        └─ ... (put your files here)
```